Softmax Regression

1. Introduction & Concept

Softmax Regression is a generalization of logistic regression used for **multi-class classification**. While binary classification handles two classes (0 or 1), Softmax handles C distinct classes.

- **Goal:** Recognize one of C classes given an input x.
- **Notation:**
  - C: The total number of classes.
  - Classes are indexed from 0 to $C-1$.
  - **Example:** If recognizing cats, dogs, chicks, and "other":
    - $C = 4$.
    - Labels: 0 (Other), 1 (Cat), 2 (Dog), 3 (Chick).

2. Neural Network Architecture

In a neural network performing multi-class classification, the output layer (Layer L) changes to accommodate the multiple classes.

- **Output Units:** The number of units in the final layer, $n^{[L]}$, equals C.
- **Output Vector:** The output $\hat{y}$ (or $a^{[L]}$) is a dimensional vector of size $(C, 1)$.
- **Probabilities:** Each node in the output layer represents the probability that the input belongs to a specific class.
  - $P(y = i|x)$ for $i = 0, \ldots, C-1$.
  - **Constraint:** The sum of all elements in the output vector $\hat{y}$ must equal 1.

3. The Softmax Activation Function

The Softmax layer computes the prediction in two main steps: the linear computation followed by the Softmax activation.

Step 1: Linear Computation

Compute the linear portion for the final layer L as usual:
$z^{[L]} = W^{[L]}a^{[L-1]} + b^{[L]}$

- Where $z^{[L]}$ is a $(C, 1)$ vector.

Step 2: Softmax Activation

The activation function is applied to the vector $z^{[L]}$ to obtain the normalized probability vector $a^{[L]}$.

1. **Exponentiation (Element-wise):** Compute a temporary vector t where each element is e raised to the power of the corresponding z value.
   $t = e^{z^{[L]}}$
2. **Normalization:** Divide each element of t by the sum of all elements in t to ensure they sum to 1.
   $a_i^{[L]} = \frac{t_i}{\sum_{j=1}^{C} t_j} = \frac{e^{z_i^{[L]}}}{\sum_{j=1}^{C} e^{z_j^{[L]}}}$

Key Characteristic: unlike typical activation functions (e.g., Sigmoid, ReLU) that map a real number to a real number, the **Softmax activation** maps a vector to a vector, as normalization requires information from all output nodes simultaneously.

4. Numerical Example

Assume $C = 4$ and the computed linear vector $z^{[L]} = [5, 2, -1, 3]^T$.

1. **Compute Exponentials (t):**
   - $e^5 \approx 148.4$
   - $e^2 \approx 7.4$
   - $e^{-1} \approx 0.4$
   - $e^3 \approx 20.1$
2. **Sum:** $148.4 + 7.4 + 0.4 + 20.1 \approx 176.3$
3. **Normalize ($a^{[L]}$):**
   - $y_0 = 148.4/176.3 \approx 0.842$ (84.2%)
   - $y_1 = 7.4/176.3 \approx 0.042$ (4.2%)
   - $y_2 = 0.4/176.3 \approx 0.002$ (0.2%)
   - $y_3 = 20.1/176.3 \approx 0.114$ (11.4%)

5. Decision Boundaries

- **No Hidden Layers:** If Softmax is applied directly to raw inputs (no hidden layers), the decision boundaries between any two classes are **linear**. It segments the input space using linear boundaries to separate multiple classes.
- **Deep Networks:** When combined with hidden layers, the network can learn complex, **non-linear** decision boundaries to separate multiple classes.

Training a Softmax Classifier

1. Intuition and Properties

- **Softmax vs. Hardmax:**
  - **Hardmax:** A mapping that looks at the vector z and assigns a value of 1 to the largest element and 0 to all others. It is a strict, binary selection.
  - **Softmax:** A "gentler" mapping that converts raw scores (z) into probabilities, preserving the relative magnitudes of the inputs as probabilities that sum to 1.
- **Generalization of Logistic Regression:**
  - Softmax Regression is the generalization of Logistic Regression to $C > 2$ classes.
  - If $C = 2$, Softmax Regression reduces mathematically to Logistic Regression. The two outputs (p and $1 - p$) become redundant, effectively requiring only one computation.

2. Loss Function (Single Training Example)

To train the network, we must define a loss function that measures the discrepancy between the target output and the predicted probability.

- **Target (y):** A one-hot encoded vector of size $(C, 1)$.
  - *Example:* If Class 2 is the true class (out of 4), $y = [0, 1, 0, 0]^T$.
- **Prediction ($\hat{y}$):** The output vector $a^{[L]}$ containing probabilities.
- **Loss Formula:**
  $L(\hat{y}, y) = -\sum_{j=1}^{C} y_j \log(\hat{y}_j)$
- **Simplification:**
  - Since $y_j = 0$ for all classes except the true class, the summation collapses to a single term.
  - If the true class is k (where $y_k = 1$), the loss becomes:
    $L(\hat{y}, y) = -\log(\hat{y}_k)$
- **Objective:**
  - To minimize the loss, the algorithm must minimize $-\log(\hat{y}_k)$.
  - This is equivalent to **maximizing** $\hat{y}_k$ (the probability assigned to the correct class).
  - This approach corresponds to **Maximum Likelihood Estimation** in statistics.

3. Cost Function (Entire Training Set)

The cost function J for the entire training set of m examples is the average of the individual losses.

$J(W, b) = \frac{1}{m} \sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)})$

4. Implementation Details

- **Matrix Dimensions:**
  - If you have m training examples and C classes:
  - The Ground Truth Matrix Y is (C, m).
  - The Prediction Matrix $\hat{Y}$ is (C, m).
  - These are constructed by stacking individual vectors $y^{(i)}$ and $\hat{y}^{(i)}$ horizontally.
- **Gradient Descent (Backpropagation):**
  - To initialize backpropagation, we need the derivative of the cost with respect to the linear output of the final layer ($z^{[L]}$).
  - **Key Derivative Formula:**
    $dz^{[L]} = \hat{y} - y$
    (or in notation: $a^{[L]} - y$)
  - This derivative is a $(C, 1)$ vector for a single example (or (C, m) for vectorized batch processing).
  - **Note:** When using Deep Learning Programming Frameworks (e.g., TensorFlow, PyTorch), you typically only define the **forward propagation**. The framework automatically handles the derivation and backward pass.
